可视计算与交互概论
Lab 4 (Animation)
实验报告

2025 年 11 月

1 Task 1 : Inverse Kinematics

Sub-Task 1

实现前向运动学 (forward kinematic) 算法

代码实现

```
void ForwardKinematics(IKSystem & ik, int StartIndex) {
    if (StartIndex == 0) {
        ik.JointGlobalRotation[0] = ik.JointLocalRotation[0];
        ik.JointGlobalPosition[0] = ik.JointLocalOffset[0];
        StartIndex                = 1;
    }

    for (int i = StartIndex; i < ik.JointLocalOffset.size(); i++) {
        // your code here: forward kinematics, update JointGlobalPosition and JointGlobalRotation
        ik.JointGlobalPosition[i] = ik.JointGlobalPosition[i - 1]
        + ik.JointGlobalRotation[i - 1] * ik.JointLocalOffset[i];
        ik.JointGlobalRotation[i] = glm::normalize(ik.JointGlobalRotation[i - 1] * ik.JointLocalRotation[i]);
    }
}
```

Sub-Task 2

实现 CCD IK 算法

代码实现

```
void InverseKinematicsCCD(IKSystem & ik, const glm::vec3 & EndPosition, int maxCCDIKIteration, float eps) {
    ForwardKinematics(ik, 0);
    // These functions will be useful: glm::normalize, glm::rotation, glm::quat * glm::quat
    for (int CCDIKIteration = 0; CCDIKIteration < maxCCDIKIteration && glm::l2Norm(ik.EndEffectorPosition() - EndPosition) > eps; CCDIKIteration++) {
        // your code here: ccd ik
        int n = ik.NumJoints();
        for (int i = n - 2; i >= 0; i--) {
            glm::vec3 jointPos = ik.JointGlobalPosition[i];
            glm::vec3 currentEndPos = ik.EndEffectorPosition();

            glm::vec3 toCurrentEnd  = glm::normalize(currentEndPos - jointPos);
            glm::vec3 toTarget      = glm::normalize(EndPosition - jointPos);
            glm::quat rotation_quat = glm::rotation(toCurrentEnd, toTarget);
            ik.JointLocalRotation[i]  = glm::normalize(rotation_quat *
            ik.JointLocalRotation[i]);
            ForwardKinematics(ik, i);
        }
    }
}
```

Sub-Task 3

实现 FABR IK 算法

代码实现

```
void InverseKinematicsFABR(IKSystem & ik, const glm::vec3 & EndPosition, int maxFABRIKIteration, float eps) {
    ForwardKinematics(ik, 0);
    int nJoints = ik.NumJoints();
    std::vector<glm::vec3> backward_positions(nJoints, glm::vec3(0, 0, 0)), forward_positions(nJoints, glm::vec3(0, 0, 0));
    for (int IKIteration = 0; IKIteration < maxFABRIKIteration && glm::l2Norm(ik.EndEffectorPosition() - EndPosition) > eps; IKIteration++) {
        // task: fabr ik
        // backward update
        glm::vec3 next_position         = EndPosition;
        backward_positions[nJoints - 1] = EndPosition;

        for (int i = nJoints - 2; i >= 0; i--) {
            // your code here
            glm::vec3 dir = glm::normalize(ik.JointGlobalPosition[i] - next_position);
            float length = glm::l2Norm(ik.JointLocalOffset[i + 1]);
            backward_positions[i] = next_position + dir * length;
            next_position = backward_positions[i];
        }

        // forward update
        glm::vec3 now_position = ik.JointGlobalPosition[0];
        forward_positions[0] = ik.JointGlobalPosition[0];
        for (int i = 0; i < nJoints - 1; i++) {
            // your code here
            glm::vec3 dir = glm::normalize(backward_positions[i + 1] -
            now_position);
            float length = glm::l2Norm(ik.JointLocalOffset[i + 1]);
            forward_positions[i + 1] = now_position + dir * length;
            now_position = forward_positions[i + 1];
        }
        ik.JointGlobalPosition = forward_positions; // copy forward positions to joint_positions
    }

    // Compute joint rotation by position here.
    for (int i = 0; i < nJoints - 1; i++) {
        ik.JointGlobalRotation[i] = glm::rotation(glm::normalize(
        ik.JointLocalOffset[i + 1]), glm::normalize(ik.JointGlobalPosition[i + 1] - ik.JointGlobalPosition[i]));
    }
    ik.JointLocalRotation[0] = ik.JointGlobalRotation[0];
```

```
    for (int i = 1; i < nJoints - 1; i++) {
        ik.JointLocalRotation[i] = glm::inverse(ik.JointGlobalRotation[i - 1]) *
        ik.JointGlobalRotation[i];
    }
    ForwardKinematics(ik, 0);
}
```

实现效果图

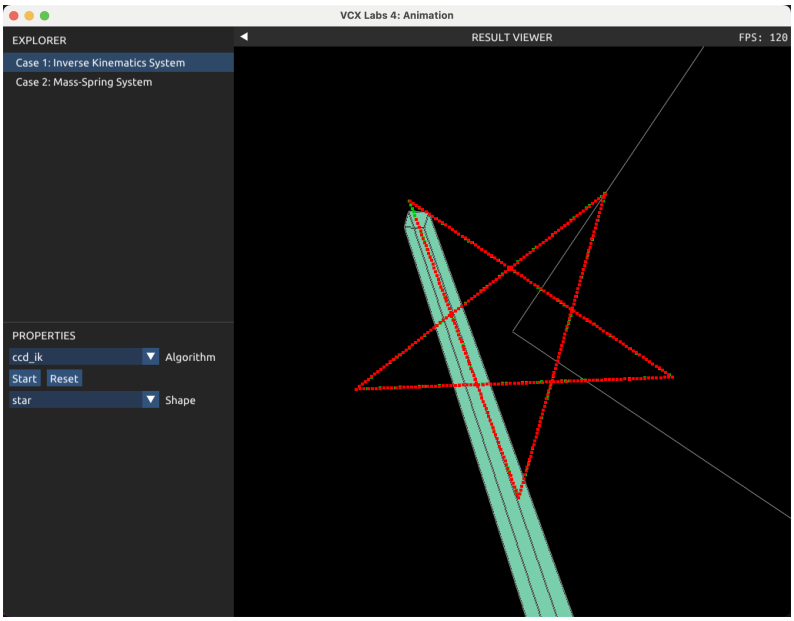


图 1: star 实现效果图

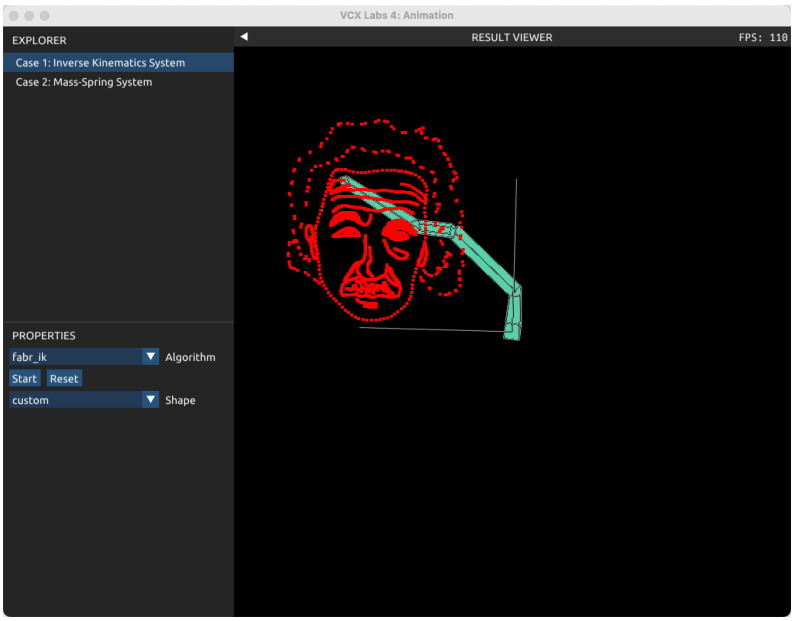


图 2: Albert Einstein curve 实现效果图

Sub-Task 4

使用 IK 绘制自定义曲线。程序入口在 tasks.cpp 中 BuildCustomTargetPosition 函数。模仿原有的函数进行修改，返回类型为 std::shared_ptr<std::vector<glm::vec3»，自定义绘制曲线（比如数字，姓名首字母缩写等）

自定义绘制“KEM”

实现效果图

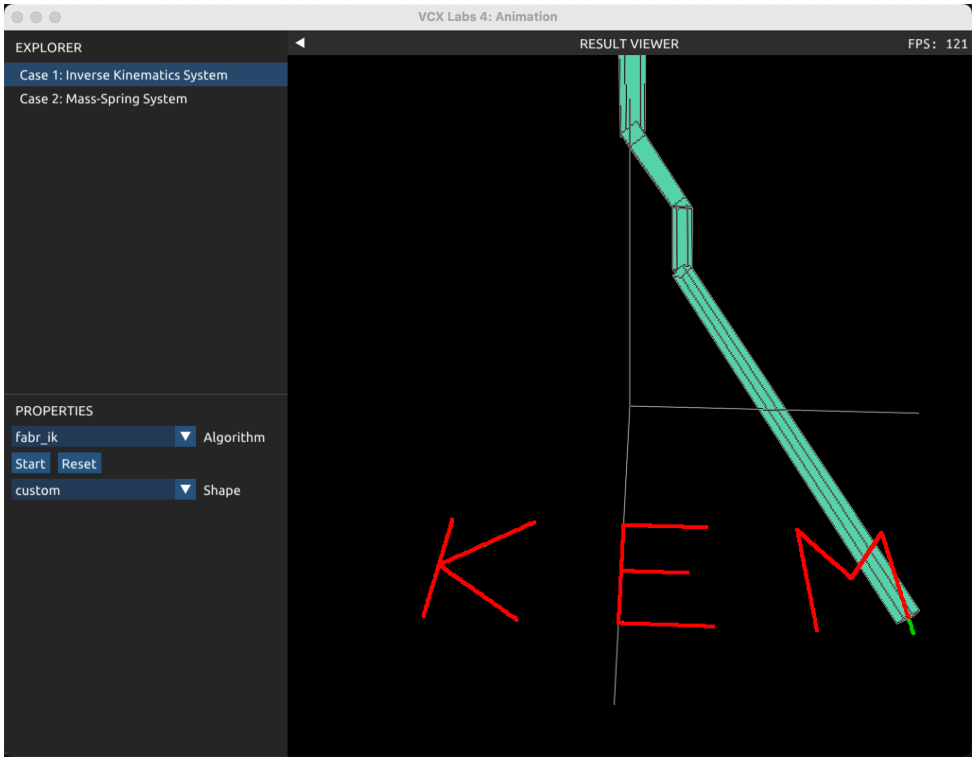


图 3: 实现效果图

Sub-Task 4.1(Bonus 任务)

在通过函数生成轨迹时，会出现采样点不均匀的地方。比如在这个图里，头像的头发非常稀疏。有无可能让采样点更加均匀?

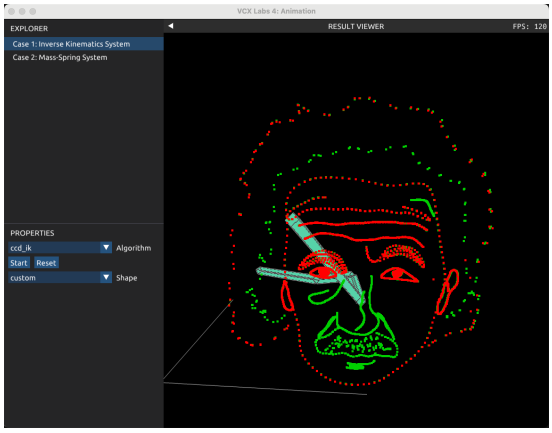
解答

有几种方案可以让采样点更加均匀:

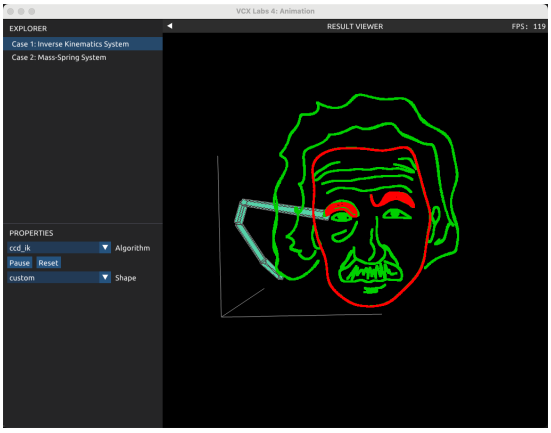
- 方法一：增加采样密度
- 方法二：弧长参数化
  - 当参数 t 均匀变化时，曲线上的实际点间距可能很不均匀，曲率大的地方点很稀疏；平缓的地方点很密集
  - 基于弧长可以使曲线上相邻点的距离基本相等，实现均匀分布

实现效果图

使用方法一



(a) nums = 5000



(b) nums = 50000

Sub-Task 4.2(Bonus 任务)

输入一张简单的 2D 图像（比如 MNIST 手写数字），提取图像的骨架，作为 IK 的目标轨迹。

代码思路

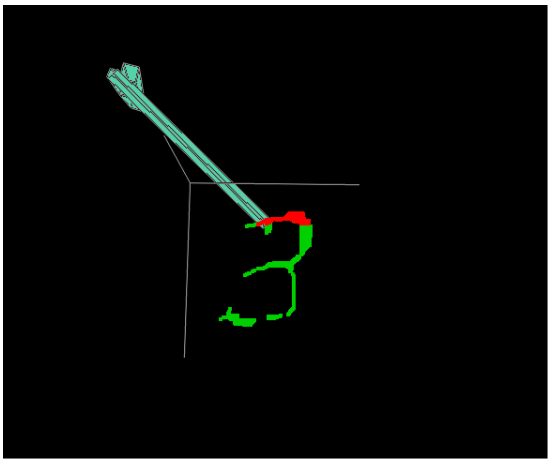
```
std::vector<glm::vec3> LoadSimpleContourFromImage(const std::string& imagePath) {
    std::vector<glm::vec3> points;

    // 读取图像
    SimpleImage input = LoadImageFile(imagePath);

    // 提取图像骨架
    SimpleImage skeleton = ExtractSkeleton(input);

    // 提取轮廓点
    points = ExtractContourPoints(skeleton);

    return points;
}
```

其中

- LoadImageFile：读取图像文件并转换为灰度图
- ExtractSkeleton：通过迭代细化算法提取图像的中心骨架
- ExtractContourPoints：从骨架图像中采样生成 3D 轨迹点
- LoadSimpleContourFromImage：整合整个流程，从图像生成机械臂运动轨迹点

实现效果图

(a) 输入图



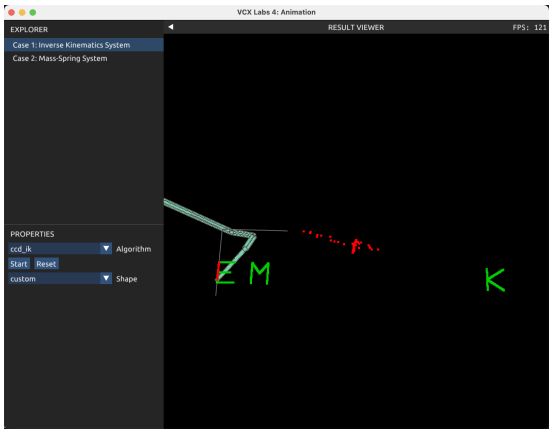
(b) 效果图

回答问题

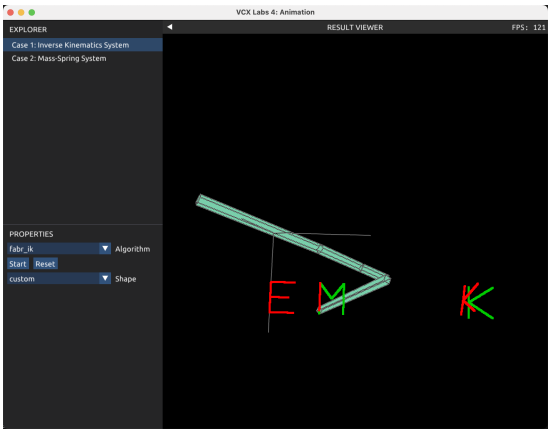
1. 如果目标位置太远，无法到达，IK 结果会怎样？

解答

两种 IK 都会尽可能伸展所有关节指向目标方向，在最大臂长范围内尽可能接近目标
CCD IK 由于单向收敛，可能产生不自然的“螺旋状”或“蛇形”构型的散点
FABR IK 由于是双向调整，会比 CCD 更稳定地收敛到拉伸状态，所以会在可以达到的最远处画出对应图像，保持更好的几何一致性



(a) CCD IK



(b) FABR IK

2. 比较 CCD IK 和 FABR IK 所需要的迭代次数。

解答

FABR IK 需要的迭代次数**少于** CCD IK
因为 FABR IK 是双向收敛，更稳定，可以更快地达到精确解

3. （选做，只提供大概想法即可）由于 IK 是多解问题，在个别情况下，会出现前后两帧关节旋转抖动的情况。怎样避免或是缓解这种情况？

解答

可能可行的方法：

- 在关节旋转中加入惯性
- 选择与上一帧最接近的旋转
- 限制关节旋转范围，减少解空间

2 Task 2: Mass-Spring System

你需要实现 tasks.cpp 中已经实现了一个显式 Euler 的弹簧质点系统。该方法的稳定性很差。为了达到稳定实时模拟的要求，你需要将它改写为隐式 Euler。

思路

在隐式欧拉积分中

$$x^{k+1} = x^k + hv^{k+1}$$

$$v^{k+1} = v^k + hM^{-1}f(x^{k+1})$$

联立得

$$x^{k+1} = x^k + hv^k + h^2M^{-1}f(x^k + 1)$$

代入

$$f(x^{k+1}) = f_{int}(x^{k+1}) + f_{ext}$$

结合上式，可以得到

$$x^{k+1} = (x^k + hv^k + h^2M^{-1}f_{ext}) + h^2M^{-1}f_{int}(x^{k+1})$$

记 $y^k = x^k + hv^k + h^2M^{-1}f_{ext}$ 得到方程组

$$x^{k+1} - y^k - h^2M^{-1}f_{int}(x^{k+1}) = x^{k+1} - y^k + h^2M^{-1}\frac{\partial E}{\partial x}(x^{k+1}) = 0$$

它等价于求解如下的优化问题

$$x^{k+1} = arg_x min \frac{1}{2h^2}||x - y^k||_M^2 + E(x)$$

设需要最小化的目标能量函数为

$$g(x) = \frac{1}{2h^2}||x - y^k||_M^2 + E(x)$$

对目标函数在当前尝试解 x_i 处进行二阶泰勒展开：

$$g(x) = g(x_i) + \nabla g(x_i) \cdot (x - x_i) + \frac{1}{2}(x - x_i)^T H_g(x_i)(x - x_i) + O(||x - x_i||^3)$$

得到了一个用于近似 g(x) 的二次函数 (忽略三阶小量)，其中 H_g 是 g 的海瑟矩阵
于是可以使用牛顿法进行迭代，多次迭代后得到一个稳定解，作为全局最小值点的近似
对上式两边求梯度，再代入 x_{i+1} 可得

$$\nabla g(x_{i+1}) \approx \nabla g(x_i) + H_g(x_i)(x_{i+1} - x_i) = 0$$

依此求得下一轮的尝试解

$$x_{i+1} = x_i - H_g^{-1}(x_i)\nabla g(x_i)$$

由 g(x) 的定义得

$$\nabla g(x^k) = \frac{1}{h^2}M(x^k - y^k) + \nabla E(x^k)$$

$$H_g(x^k) = \frac{1}{h^2}M + H(x^k)$$

其中 $H(x^k)$ 是弹性势能 E(x) 的海瑟矩阵
考虑一个弹簧 (i, j) 关于质点 i 的海瑟矩阵

$$H_e := \frac{\partial^2 E_{ij}(x^k)}{\partial x_i^2} = k_{ij}\frac{(x_i - x_j)(x_i - x_j)^T}{||x_i - x_j||^2} + k_{ij}(1 - \frac{l_{ij}}{||x_i - x_j||})(I - \frac{(x_i - x_j)(x_i - x_j)^T}{||x_i - x_j||^2})$$

由此可以得到 $H_{ij}(x^k)$
总能量的海瑟矩阵即为所有弹簧海瑟矩阵之和

$$H(x^k) = \sum_{(i,j)} H_{ij}(x^k)$$

于是可以计算 $\nabla g(x^k)$ $H_g(x^k)$
即可求解方程 $x_{i+1} = x_i - H_g^{-1}(x_i)\nabla g(x_i)$

代码实现

计算内力 f_{ij} 的函数

```
Eigen::VectorXf f_inner(MassSpringSystem & system, Eigen::VectorXf & x_vec){
    std::vector<glm::vec3> x_vec3 = eigen2glm(x_vec);
    int n = system.Positions.size();
    std::vector<glm::vec3> forces(n, glm::vec3(0, 0, 0));

    for (auto const & spring : system.Springs) {
        auto const p0 = spring.AdjIdx.first;
        auto const p1 = spring.AdjIdx.second;

        glm::vec3 const x01 = x_vec3[p1] - x_vec3[p0];
        float length = glm::length(x01);

        glm::vec3 e01 = glm::normalize(x01);
        glm::vec3 f = system.Stiffness * (length - spring.RestLength) * e01;
        forces[p0] += f;
        forces[p1] -= f;
    }
    return glm2eigen(forces);
}
```

计算 g 的梯度的函数

```
Eigen::VectorXf grad_g(MassSpringSystem & system, Eigen::VectorXf & x_vec, Eigen::
    VectorXf & y_vec, float ddt){
    int n = system.Positions.size();
    Eigen::VectorXf grad = Eigen::VectorXf::Zero(3 * n);

    // 惯性部分的梯度
    grad += system.Mass / (ddt * ddt) * (x_vec - y_vec);

    // 弹簧部分的梯度
```

```
    grad -= f_inner(system, x_vec);

    return grad;
}
```

计算海瑟矩阵的函数

```
void Hessian_Matrix(MassSpringSystem & system, Eigen::VectorXf & x_vec, float ddt,
    Eigen::SparseMatrix<float> & dst_matrix){
    int n = system.Positions.size();
    std::vector<Eigen::Triplet<float>> triplets;

    // 质量矩阵
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < 3; j++) {
            triplets.emplace_back(3 * i + j, 3 * i + j, system.Mass / (ddt * ddt));
        }
    }

    // 弹簧刚度矩阵
    std::vector<glm::vec3> x_vec3 = eigen2glm(x_vec);
    for (auto const & spring : system.Springs) {
        auto const p0 = spring.AdjIdx.first;
        auto const p1 = spring.AdjIdx.second;

        glm::vec3 const x01 = x_vec3[p1] - x_vec3[p0];
        float length = glm::length(x01);
        glm::vec3 e01 = glm::normalize(x01);
        float k = system.Stiffness * (1 - spring.RestLength / length);
        glm::mat3 outer_product;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                outer_product[i][j] = e01[i] * e01[j];
            }
        }

        glm::mat3 I = glm::mat3(1.0f);

        glm::mat3 H_e = system.Stiffness * (outer_product + (1.0f - spring.
            RestLength / length) * (I - outer_product));

        for (int a = 0; a < 3; a++) {
            for (int b = 0; b < 3; b++) {
                triplets.emplace_back(3 * p0 + a, 3 * p0 + b, H_e[a][b]);
                triplets.emplace_back(3 * p1 + a, 3 * p1 + b, H_e[a][b]);
                triplets.emplace_back(3 * p0 + a, 3 * p1 + b, -H_e[a][b]);
                triplets.emplace_back(3 * p1 + a, 3 * p0 + b, -H_e[a][b]);
            }
        }
```

```
    }
    dst_matrix = CreateEigenSparseMatrix(3 * n, triplets);
}
```

使用隐式 Euler 方法的弹簧质点系统函数

```
void AdvanceMassSpringSystem(MassSpringSystem & system, float const dt) {
    // your code here: rewrite following code

    // use implicit euler method to advance mass spring system
    int const steps = 1;
    float const ddt = dt / steps;
    for (std::size_t s = 0; s < steps; s++) {
        int n = system.Positions.size();
        Eigen::VectorXf x0 = glm2eigen(system.Positions);
        Eigen::VectorXf v0 = glm2eigen(system.Velocities);
        Eigen::VectorXf f0 = glm2eigen(std::vector<glm::vec3>(n, glm::vec3(0, -
            system.Gravity * system.Mass, 0)));

        Eigen::VectorXf y0 = x0 + ddt * v0 + ddt * ddt * f0 / system.Mass;

        // 计算梯度
        Eigen::VectorXf g_grad = grad_g(system, x0, y0, ddt);

        // 计算Hessian矩阵
        Eigen::SparseMatrix<float> H;
        Hessian_Matrix(system, x0, ddt, H);

        // 求解步长
        Eigen::VectorXf delta_x = ComputeSimplicialLLT(H, -g_grad);

        // 更新位置和速度
        Eigen::VectorXf x1 = x0 + delta_x;
        Eigen::VectorXf inner_force = f_inner(system, x1);
        Eigen::VectorXf v1 = v0 + ddt * (inner_force + f0) / system.Mass;

        // 更新系统状态
        std::vector<glm::vec3> new_positions = eigen2glm(x1);
        std::vector<glm::vec3> new_velocities = eigen2glm(v1);
        for (int i = 0; i < n; i++) {
            if (!system.Fixed[i]) {
                system.Positions[i] = new_positions[i];
                system.Velocities[i] = new_velocities[i];
            }
        }
    }
}
```

实现效果图

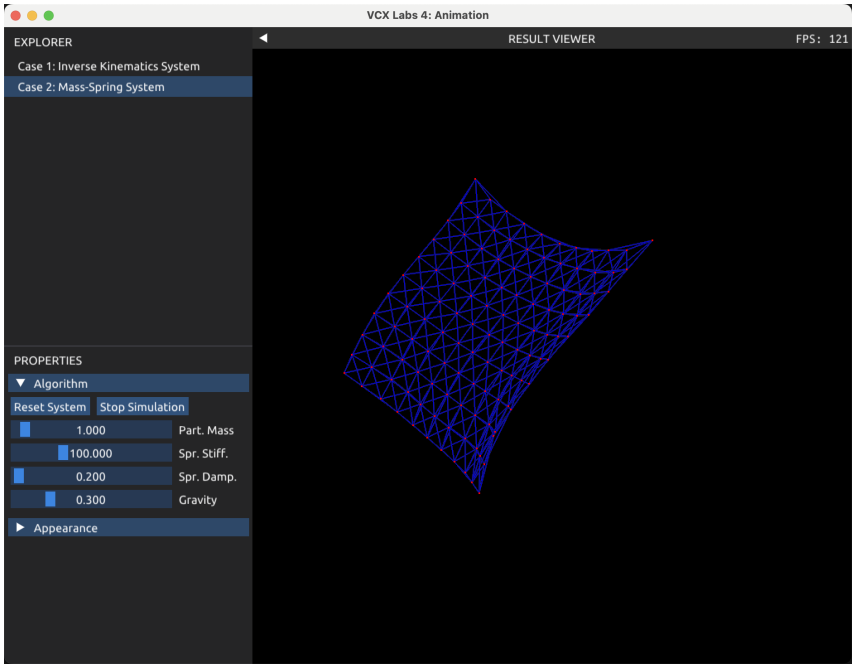


图 7: 实现效果图

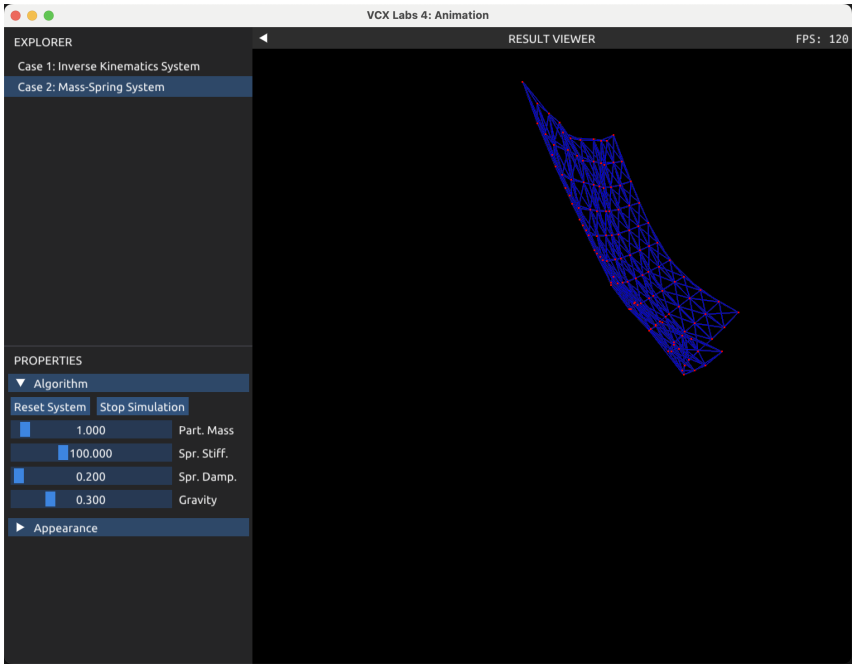


图 8: 实现效果图